FORMACIÓN PROFESIONAL · GRADO SUPERIOR

DAM 2º

UT1 · Manejo de Ficheros

Texto, binarios y XML en Java: leer, escribir y convertir información persistente

MÓDULO
Acceso a Datos (0486)

UNIDAD
UT1 / UD01

RESULTADO DE APRENDIZAJE
RA1 · CE1.a–CE1.g

CURSO
2026-2027 · Sergio Capdevila Díez

📑 ÍNDICE DE CONTENIDOS

1. Introducción
2. Fundamentos comunes
3. Ficheros de texto (java.io y NIO.2)
4. Ficheros binarios y serialización
5. Ficheros XML (DOM, SAX, StAX y JAXB)
6. Integración: aplicación multi-formato
7. Visión de conjunto
8. Resumen y conceptos que debes dominar
9. Glosario y preguntas de repaso
10. Anexo: trazabilidad

1 Introducción: guardar información en ficheros

Casi ningún programa vive solo en la memoria. En cuanto cierras la aplicación, todo lo que había en la RAM desaparece: las variables, los objetos, las listas. Si queremos que un dato siga ahí mañana — una configuración, un catálogo de productos, un registro de operaciones — hay que **guardarlo en un fichero** del disco. A eso lo llamamos *persistencia*.

En esta unidad aprenderás a leer y escribir ficheros en Java de las tres formas que usa la industria: **texto** (lo que abres con el Bloc de notas), **binario** (datos compactos que solo entiende el programa) y **XML** (el formato estándar para intercambiar información entre empresas y sistemas). Al terminar

sabrás elegir la herramienta adecuada para cada caso y construir una pequeña aplicación que mueva datos entre los tres formatos.

> **🏭 ¿Dónde se usa esto en la empresa?**
>
> Un TPV exporta las ventas del día a un `.csv`; una máquina industrial graba su telemetría en un binario compacto para ahorrar espacio; una gestoría recibe las facturas de sus clientes en XML (Facturae); una aplicación guarda tus preferencias en un fichero de configuración. Todo eso es lo que vas a saber programar.

Resultado de aprendizaje y criterios de evaluación

Esta unidad desarrolla por completo el **RA1**: *«Desarrolla aplicaciones que gestionan información almacenada en ficheros identificando el campo de aplicación de los mismos y utilizando clases específicas»*. Sus siete criterios de evaluación son:

CE	QUÉ SE EVALÚA
CE1.a	Usar clases para la gestión de ficheros y directorios.
CE1.b	Valorar ventajas e inconvenientes de las distintas formas de acceso.
CE1.c	Usar clases para *recuperar* información almacenada en ficheros.
CE1.d	Usar clases para *almacenar* información en ficheros.
CE1.e	Realizar conversiones entre diferentes formatos de ficheros.
CE1.f	Prever y gestionar las excepciones.
CE1.g	Probar y documentar las aplicaciones desarrolladas.

Qué necesitas saber antes de empezar

Estos apuntes son autosuficientes para la unidad, pero damos por sabido de primer curso:

- Programación orientada a objetos en Java: clases, atributos, constructores, métodos y `toString()`.
- Tipos primitivos (`int`, `double`, `boolean`, `char`) y cadenas (`String`).
- Manejo básico de excepciones con `try` / `catch`.
- Colecciones y, para el último bloque, nociones de *streams* y expresiones lambda.

Cómo vamos a trabajar

La unidad avanza de lo general a lo concreto. Primero los **fundamentos comunes** a todos los ficheros; después un bloque por cada formato; al final, un caso que lo combina todo. Cada apartado teórico tiene su **actividad de aula** (la programas en tu IDE mientras se explica) y, al cerrar cada bloque, quedan desbloqueados los ejercicios de refuerzo. El recorrido es:

1. **Ficheros de texto** (java.io y NIO.2) · actividades 1.1 a 1.6 · ejercicios del bloque 1.
2. **Ficheros binarios y serialización** · actividades 2.1 a 2.5 · ejercicios del bloque 2.
3. **Ficheros XML** (DOM, SAX, StAX y JAXB) · actividades 3.1 a 3.5 · ejercicios del bloque 3.
4. **Integración**: una aplicación que convierte datos entre formatos, probada y documentada · práctica final.

2 Fundamentos comunes

Antes de entrar en cada formato conviene fijar cinco ideas que vas a usar en toda la unidad, sea el fichero de texto, binario o XML: qué es un fichero y cómo se accede a él, cómo se nombran las rutas, qué es un flujo de datos, por qué usamos un búfer y cómo se liberan los recursos de forma segura.

2.1 Qué es un fichero y formas de acceso

Un **fichero** es una secuencia de bytes guardada en un dispositivo (disco duro, SSD, memoria USB) con un nombre. Para el sistema operativo no es más que eso: una tira de bytes, del primero al último. Lo que cambia es *cómo interpretamos* esos bytes y *en qué orden* los recorremos.

Según **qué representan** los bytes, distinguimos dos grandes familias:

- **Ficheros de texto**: cada byte (o grupo de bytes) representa un carácter legible según una tabla de codificación. Los abres con cualquier editor y los entiendes: `.txt`, `.csv`, `.xml`, `.json`, `.log`.
- **Ficheros binarios**: los bytes son la representación de máquina de los datos (un `int` ocupa 4 bytes, un `double` 8). Ocupan menos y se leen más rápido, pero si los abres con el Bloc de notas ves símbolos ilegibles.

Según **en qué orden** recorremos los bytes, hay dos formas de acceso:

- **Acceso secuencial**: se lee o escribe del principio al final, en orden. Para llegar al registro 500 hay que pasar por los 499 anteriores. Es lo normal en texto y en los flujos binarios.
- **Acceso aleatorio** (o directo): tratamos el fichero como un gran array de bytes y saltamos a cualquier posición con un puntero. Permite modificar un dato en mitad del fichero sin reescribir el resto. Lo veremos con `RandomAccessFile` (apartado 4.3).

> 💡 **La idea que decide el diseño**
>
> La primera pregunta ante cualquier problema de ficheros es: *¿los datos son para que los lea una persona o solo el programa?* y *¿necesito llegar directo a un registro concreto o me vale recorrerlo entero?*. Esas dos respuestas (texto/binario y secuencial/aleatorio) determinan qué clases de Java vas a usar. Este es el criterio **CE1.b**.

2.2 Rutas, ficheros y directorios: `File`, `Path` y `Files`

Para trabajar con un fichero primero hay que **nombrarlo**: indicar dónde está. Esa dirección es la *ruta*. Aquí hay una trampa clásica: si escribes la ruta a mano con barras de Windows (`"C:\\datos\\archivo.txt"`), tu programa dejará de funcionar en Linux o en el ordenador del profesor. La solución es construir las rutas con las clases de Java, que ponen el separador correcto en cada sistema.

Java tiene dos generaciones de clases para esto. La clásica, `java.io.File`, todavía se usa mucho. La moderna, **NIO.2** (desde Java 7), gira en torno a `java.nio.file.Path` (una ruta) y `java.nio.file.Files` (una caja de métodos estáticos para operar con ellas). NIO.2 es preferible: cuando algo falla, lanza una excepción que explica el motivo, en vez del `false` silencioso de `File`.

INSTRUCCIÓN	DEVUELVE	QUÉ HACE
`Path.of("datos", "a.txt")`	`Path`	Construye la ruta uniendo nombres con el separador del sistema.
`ruta.resolve("sub")`	`Path`	Añade un tramo a la ruta.
`ruta.getFileName()` / `getParent()`	`Path`	Nombre del fichero / carpeta que lo contiene.
`Files.exists(ruta)` / `isDirectory(ruta)`	`boolean`	Comprueba si existe / si es una carpeta.
`Files.createDirectories(ruta)`	`Path`	Crea toda la cadena de carpetas que falte. No falla si ya existen.
`Files.createFile(ruta)`	`Path`	Crea un fichero vacío (falla si ya existe).
`ruta.toFile().listFiles()`	`File[]`	Lista el contenido de una carpeta.
`Files.move(o, d, REPLACE_EXISTING)`	`Path`	Mueve o renombra un fichero.
`Files.deleteIfExists(ruta)`	`boolean`	Borra si existe; devuelve `true` si lo borró.
`Files.size(ruta)`	`long`	Tamaño en bytes.

GestionDirectorios.java JAVA

```java
import java.nio.file.Files;
import java.nio.file.Path;

public class GestionDirectorios {
    public static void main(String[] args) throws Exception {
        // 1. La ruta es solo un nombre: todavía no existe nada en el disco
        Path mes = Path.of("datos", "exportaciones", "2026", "09");

        // 2. createDirectories crea TODA la cadena que falte (2026 y 09)
        Files.createDirectories(mes);
        System.out.println("Carpeta creada: " + mes.toAbsolutePath());

        // 3. Crear un fichero dentro, solo si no existe
        Path informe = mes.resolve("informe.txt");
        if (!Files.exists(informe)) {
            Files.createFile(informe);
        }

        // 4. Comprobaciones
        System.out.println("El informe existe: " + Files.exists(informe));
        System.out.println("'exportaciones' es carpeta: "
                + Files.isDirectory(Path.of("datos", "exportaciones")));
    }
}
```

Paso a paso: `Path.of(...)` solo arma el nombre; nada toca el disco todavía. `Files.createDirectories(mes)` crea `datos`, `exportaciones`, `2026` y `09` de una vez, y no protesta si alguna ya estaba. Solo creamos el fichero si no existe, porque `createFile` lanzaría `FileAlreadyExistsException` sobre uno que ya está. Este es el criterio **CE1.a**: gestionar ficheros y directorios.

> ⚠️ **Error habitual**
>
> Confundir `Files.createDirectory` (singular: crea solo el último nivel y falla si falta un intermedio) con `Files.createDirectories` (plural: crea toda la cadena). Para rutas de varios niveles, casi siempre quieres el plural.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 0.1** del cuaderno de actividades de aula: organizar una carpeta de exportaciones creando, listando, moviendo y borrando ficheros.

2.3 Flujos (streams) de bytes y de caracteres

Java no lee un fichero «de golpe»: lo trata como un **flujo** (*stream*) de datos que va del disco a tu programa (lectura) o de tu programa al disco (escritura). Imagina una tubería por la que circulan los

datos en fila. Según lo que circule por la tubería, hay dos jerarquías de clases:

- **Flujos de bytes** — `InputStream` (entrada) y `OutputStream` (salida). Transportan bytes puros. Se usan para binario: imágenes, `.dat`, objetos serializados.
- **Flujos de caracteres** — `Reader` (entrada) y `Writer` (salida). Transportan caracteres ya traducidos desde los bytes según una codificación. Se usan para texto.

Esos cuatro nombres (`InputStream`, `OutputStream`, `Reader`, `Writer`) son clases *abstractas*: no se usan directamente, sino sus hijas concretas. `FileReader` y `FileWriter` conectan con un fichero de texto; `FileInputStream` y `FileOutputStream`, con uno binario. Recordar la pareja correcta es fácil: **Reader/Writer para leer y escribir texto; InputStream/OutputStream para bytes**. Recuperar datos de un fichero es el criterio **CE1.c**; almacenarlos, el **CE1.d**.

2.4 Buffer y patrón decorador

Acceder al disco para leer o escribir *un solo carácter* es carísimo: cada operación implica pedirle permiso al sistema operativo. Hacerlo carácter a carácter sería como ir a por la compra y traer un solo producto en cada viaje.

La solución es el **búfer**: una zona de memoria RAM (por defecto 8 KB) que acumula datos y los vuelca al disco en bloques grandes. En Java esto se consigue con el **patrón decorador**: se envuelve un flujo básico dentro de otro que le añade el búfer.

El patrón decorador — TEXTO

```
[ Disco ] <-> [ FileReader ] <-> [ BufferedReader (búfer 8 KB) ] <-> [ tu programa ]
   bytes        traduce a           acumula en RAM y                lee cómodo
   crudos       caracteres          entrega línea a línea           línea a línea
```

Por eso casi siempre verás construcciones «en sándwich»: `new BufferedReader(new FileReader(...))`. El `FileReader` hace la conexión física; el `BufferedReader` le pone el búfer y añade el cómodo método `readLine()`. Lo mismo con `BufferedWriter` sobre `FileWriter`, o `BufferedInputStream` / `BufferedOutputStream` sobre los flujos de bytes.

> 🏭 **En un entorno profesional**
>
> Envolver siempre los flujos de fichero en su versión *Buffered* no es un lujo: en ficheros grandes multiplica la velocidad. Un revisor de código te marcará un `FileReader` usado «a pelo» para leer línea a línea.

2.5 Excepciones de E/S y `try-with-resources`

Trabajar con ficheros *siempre* puede fallar: el fichero no existe, no hay permisos, el disco está lleno, se desconecta la unidad. Por eso casi todos los métodos de E/S declaran `IOException` (o hijas más concretas como `FileNotFoundException` o `NoSuchFileException`), y tu código tiene que preverla. Este es el criterio **CE1.f**.

Además, todo flujo abierto ocupa un recurso del sistema operativo (un *descriptor* de fichero) que hay que **cerrar** sí o sí. Si no lo cierras, pierdes datos (lo que quedó en el búfer no se vuelca) y en Windows el fichero queda bloqueado. Antes había que cerrarlo a mano en un `finally`, con el riesgo de olvidarlo. Hoy se usa **try-with-resources**: se declara el recurso entre paréntesis tras el `try` y Java lo cierra solo al terminar, pase lo que pase.

PatronSeguro.java JAVA

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class PatronSeguro {
    public static void main(String[] args) {
        // El recurso se declara entre paréntesis: Java llama a close() automáticamente
        try (BufferedReader br = new BufferedReader(
                new FileReader("datos/config.txt", StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = br.readLine()) != null) {   // null = fin del fichero
                System.out.println(linea);
            }
        } catch (IOException e) {                       // prevemos el fallo de E/S
            System.err.println("No se pudo leer el fichero: " + e.getMessage());
        }
        // Al salir del try, el BufferedReader ya está cerrado (haya ido bien o mal)
    }
}
```

Este patrón — `try (recurso) { ... } catch (IOException e) { ... }` — es el esqueleto de casi todo lo que viene a partir de aquí. Fíjate en el bucle `while ((linea = br.readLine()) != null)`: `readLine()` devuelve `null` cuando ya no quedan líneas, y esa es la señal canónica de fin de fichero en texto.

> ⚠️ **Error habitual**
>
> Cerrar el flujo dentro de un `finally` a mano, o no cerrarlo. Si el programa termina y el fichero de salida queda a 0 KB o incompleto, casi siempre es que faltó el cierre (y con él, el vaciado del búfer). Con try-with-resources no pasa.

3 Ficheros de texto (java.io y NIO.2)

Empezamos por los ficheros de texto porque son los más intuitivos: lo que escribes es legible. Verás cómo leer y escribir línea a línea con la biblioteca clásica `java.io`, cómo no romper las tildes con la codificación adecuada, cómo leer por campos con `Scanner` y escribir con formato con `PrintWriter`, y cómo la API moderna **NIO.2** deja casi todo esto en una línea. Este bloque trabaja los criterios **CE1.a, CE1.c, CE1.d, CE1.e** y **CE1.f**.

3.1 Leer y escribir texto línea a línea

La herramienta básica para texto son `BufferedReader` (lectura) y `BufferedWriter` (escritura), montados sobre `FileReader` y `FileWriter` como vimos en el patrón decorador. Se lee con `readLine()` hasta que devuelve `null`, y se escribe con `write(String)` seguido de `newLine()`.

MÉTODO	CLASE	QUÉ HACE
`readLine()`	`BufferedReader`	Lee una línea entera (sin el salto). Devuelve `null` al llegar al final.
`write(String s)`	`BufferedWriter`	Escribe el texto en el búfer.
`newLine()`	`BufferedWriter`	Escribe el salto de línea propio del sistema operativo.

CopiaMayusculas.java — JAVA

```java
import java.io.*;
import java.nio.charset.StandardCharsets;

public class CopiaMayusculas {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
                 new FileReader("datos/origen.txt", StandardCharsets.UTF_8));
             BufferedWriter bw = new BufferedWriter(
                 new FileWriter("datos/destino.txt", StandardCharsets.UTF_8))) {

            String linea;
            while ((linea = br.readLine()) != null) {   // null = fin del fichero
                bw.write(linea.toUpperCase());
                bw.newLine();                           // salto de línea portable
            }
            System.out.println("Copia en mayúsculas terminada.");
        } catch (IOException e) {
            System.err.println("Error de E/S: " + e.getMessage());
        }
    }
}
```

Se abren los dos flujos en el mismo `try-with-resources` (separados por `;`) y se cierran solos. El bucle lee una línea, la transforma y la escribe con su salto. Sin `newLine()`, todo el texto saldría pegado en una única línea.

> ⚠️ **Error habitual**
>
> Olvidar `bw.newLine()`: el fichero de salida queda con todas las líneas concatenadas. Es el fallo número uno al escribir texto.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 1.1** del cuaderno de aula: sanear un fichero `.ini` quitando comentarios y líneas vacías y pasando las claves a mayúsculas.

3.2 Codificación de caracteres (charsets)

Aquí está uno de los conceptos que más quebraderos de cabeza da en producción. En memoria, Java guarda los caracteres en **UTF-16**. Pero en el disco un fichero de texto son bytes, y para pasar de caracteres a bytes (y al revés) hace falta una **tabla de codificación** o *charset*:

- **ASCII**: 7 bits, 128 caracteres. No tiene tildes ni ñ.
- **ISO-8859-1** (Latin-1) y **Windows-1252**: 8 bits, 256 caracteres. Históricos en Windows europeo.
- **UTF-8**: longitud variable (1 a 4 bytes). Es el estándar de la industria: compatible con ASCII y capaz de representar cualquier idioma y los emojis.

Si escribes un fichero con un charset y lo lees con otro, las tildes y eñes se convierten en símbolos raros (`año` → `aÃ±o`). Es el fenómeno *Mojibake*. Convertir un fichero de una codificación a otra — de un sistema antiguo en ISO-8859-1 a UTF-8 — es una **conversión entre formatos**, el criterio **CE1.e**.

Para leer o escribir con un charset concreto se usan los puentes `InputStreamReader` y `OutputStreamWriter`, que traducen entre bytes y caracteres indicando la tabla:

ConversorCharset.java JAVA

```java
import java.io.*;
import java.nio.charset.StandardCharsets;

public class ConversorCharset {
    public static void main(String[] args) {
        // Leemos en ISO-8859-1 (fichero antiguo) y escribimos en UTF-8 (estándar)
        try (BufferedReader br = new BufferedReader(new InputStreamReader(
                 new FileInputStream("datos/legado.txt"), StandardCharsets.ISO_8859_1));
             BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(
                 new FileOutputStream("datos/moderno.txt"), StandardCharsets.UTF_8))) {

            String linea;
            while ((linea = br.readLine()) != null) {
                bw.write(linea);
                bw.newLine();
            }
            System.out.println("Convertido de ISO-8859-1 a UTF-8.");
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

> 💡 **Regla de oro**
>
> Indica **siempre** el charset de forma explícita: `StandardCharsets.UTF_8`. Un `new FileReader("a.txt")` sin charset usa la codificación por defecto de la máquina, que es distinta en Windows y en Linux. El código «funciona en mi ordenador» y falla en el servidor.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 1.2** del cuaderno de aula: migrar un fichero de usuarios de ISO-8859-1 a UTF-8 y comprobar que las tildes y eñes quedan bien.

3.3 Lectura por tokens con `Scanner`

A veces no queremos la línea entera, sino **cada campo** ya convertido a su tipo. Para eso está `Scanner`: lee *tokens* (trozos) separados por un delimitador y los devuelve como `String`, `int`, `double`... sin que tengas que partir la línea a mano. Es muy cómodo para leer un CSV.

MÉTODO	QUÉ HACE
`hasNextLine()` / `nextLine()`	¿Queda otra línea? / la lee entera.
`useDelimiter(";")`	Fija el separador de campos.
`useLocale(Locale.US)`	Hace que `nextDouble()` espere punto decimal (`7.5`) y no coma.
`next()` / `nextDouble()` / `nextInt()`	Lee el siguiente campo ya convertido a su tipo.

LeerCSVScanner.java JAVA

```java
import java.io.File;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Locale;
import java.util.Scanner;

public class LeerCSVScanner {
    public static void main(String[] args) {
        // productos.csv: REF;DESCRIPCION;PRECIO;UDS
        try (Scanner lector = new Scanner(new File("datos/productos.csv"),
                                          StandardCharsets.UTF_8)) {
            lector.nextLine();                          // saltamos la cabecera
            while (lector.hasNextLine()) {
                String linea = lector.nextLine();
                try (Scanner campos = new Scanner(linea)) {
                    campos.useDelimiter(";");
                    campos.useLocale(Locale.US);        // precios con punto decimal
                    String ref = campos.next();
                    String desc = campos.next();
                    double precio = campos.nextDouble();
                    int uds = campos.nextInt();
                    System.out.printf("%s (%s): %.2f x %d%n", ref, desc, precio, uds);
                }
            }
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Usamos un `Scanner` para recorrer las líneas y, por cada una, un segundo `Scanner` que la trocea por `;`. El `useLocale(Locale.US)` es clave: sin él, con la configuración española, `nextDouble()` esperaría `7,50` con coma y fallaría con `750.50`.

> **⚠️ Error habitual**
>
> Usar un solo `Scanner` con `useDelimiter(";")` para todo el fichero. El salto de línea no es un `;`, así que el último campo de una fila se pegaría al primero de la siguiente.

> **💻 Actividad de aula**
>
> Programa la **Actividad 1.3** del cuaderno de aula: leer un CSV de productos por campos con `Scanner` y sumar unidades e importe total.

3.4 Escritura con formato: `PrintWriter`

Para *escribir* texto bien presentado — una factura, un informe con columnas alineadas — la herramienta cómoda es `PrintWriter`, que añade `print()`, `println()` y sobre todo `printf()`, con las mismas máscaras de formato que `System.out.printf`.

MÁSCARA	SIGNIFICADO
`%-22s`	Cadena en 22 huecos, alineada a la **izquierda**.
`%9.2f`	Decimal en 9 huecos con 2 decimales, a la **derecha**.
`%5d`	Entero en 5 huecos, a la derecha.
`%n`	Salto de línea portable.

Factura.java JAVA

```java
import java.io.*;
import java.nio.charset.StandardCharsets;

public class Factura {
    public static void main(String[] args) {
        try (PrintWriter pw = new PrintWriter(new BufferedWriter(
                new FileWriter("datos/factura.txt", StandardCharsets.UTF_8)))) {

            pw.printf("%-9s %-22s %9s %5s%n", "REF", "DESCRIPCIÓN", "PRECIO", "UDS");
            pw.printf("%-9s %-22s %8.2f€ %5d%n", "PORT-01", "Portátil 15\"", 750.50, 2);
            pw.printf("%-9s %-22s %8.2f€ %5d%n", "RAT-02", "Ratón", 18.90, 5);

            if (pw.checkError()) {                  // PrintWriter no lanza IOException
                System.err.println("Hubo un error al escribir.");
            }
        } catch (IOException e) {
            System.err.println("No se pudo abrir: " + e.getMessage());
        }
    }
}
```

Las máscaras dejan las columnas alineadas por la coma decimal, como en una factura de verdad. Ojo a una peculiaridad: `PrintWriter` **no lanza** `IOException` al escribir; se la traga. Si sospechas un fallo de disco, tienes que preguntarlo con `checkError()`.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 1.4** del cuaderno de aula: generar un informe de ventas con columnas alineadas y el total con IVA usando `printf`.

3.5 Texto con NIO.2: `Files` y `StandardOpenOption`

Todo lo anterior con `java.io` funciona, pero es verboso. Para ficheros pequeños o medianos, **NIO.2** lo resuelve en una línea con la clase `Files`: leer o escribir un fichero entero de una vez.

MÉTODO	DEVUELVE	QUÉ HACE
`Files.writeString(ruta, texto)`	`Path`	Escribe todo el texto (crea o sobrescribe), en UTF-8.
`Files.writeString(ruta, t, APPEND)`	`Path`	Añade al final en vez de sobrescribir.
`Files.readString(ruta)`	`String`	Lee todo el fichero a un `String`.
`Files.readAllLines(ruta)`	`List<String>`	Lee todas las líneas a una lista.
`Files.newBufferedReader(ruta, cs)`	`BufferedReader`	Abre un `BufferedReader` ya montado, con charset.

ConfigNIO2.java JAVA

```java
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class ConfigNIO2 {
    public static void main(String[] args) throws Exception {
        Path ruta = Path.of("datos", "database.properties");

        // Escribir todo de una vez (crea o sobrescribe), UTF-8 por defecto
        Files.writeString(ruta, "db.host=localhost\ndb.port=3306\n");
        System.out.println("Tamaño: " + Files.size(ruta) + " bytes");

        // Añadir una línea al final sin borrar lo anterior
        Files.writeString(ruta, "db.pool=10\n", StandardOpenOption.APPEND);

        // Recuperar el contenido completo
        System.out.print(Files.readString(ruta));
    }
}
```

Tres líneas hacen lo que antes ocupaba un `try-with-resources` entero. El modificador `StandardOpenOption.APPEND` decide si añadimos o sobrescribimos: sin él, cada escritura empieza de cero. Como `Files` también gestiona rutas y directorios, este apartado toca los criterios **CE1.a, CE1.c** y **CE1.d**.

> ⚠️ **Error habitual**
>
> `Files.readString()` carga el fichero *entero* en memoria. Con un fichero de varios GB provoca `OutOfMemoryError`. Para eso está el apartado siguiente.

> **💻 Actividad de aula**
>
> Programa la **Actividad 1.5** del cuaderno de aula: crear, ampliar (APPEND) y leer un fichero de configuración con `Files`.

3.6 Procesar ficheros grandes con `Files.lines()` y streams

¿Y si el fichero es un log de 2 GB? Cargarlo entero es inviable. La solución profesional es `Files.lines()`, que devuelve un `Stream<String>` **perezoso** (*lazy*): lee las líneas del disco una a una, a medida que hacen falta, sin cargarlas todas. Sobre ese stream encadenamos operaciones funcionales (`filter`, `map`, `count`, `collect`).

AnalisisLog.java — JAVA

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class AnalisisLog {
    public static void main(String[] args) throws Exception {
        Path log = Path.of("datos", "app.log");

        // 1. Filtrar los errores y volcarlos a otro fichero
        try (Stream<String> lineas = Files.lines(log)) {     // ¡cerrar el stream!
            List<String> errores = lineas
                    .filter(l -> l.contains("[ERROR]"))
                    .toList();
            Files.write(Path.of("datos", "alertas.txt"), errores);
            System.out.println("Errores: " + errores.size());
        }

        // 2. Contar líneas por nivel con map + groupingBy
        try (Stream<String> lineas = Files.lines(log)) {
            Map<String, Long> porNivel = lineas
                    .map(l -> l.substring(l.indexOf('[') + 1, l.indexOf(']')))
                    .collect(Collectors.groupingBy(n -> n, TreeMap::new,
                                                   Collectors.counting()));
            System.out.println(porNivel);   // {DEBUG=1, ERROR=2, INFO=2}
        }
    }
}
```

`Files.lines()` abre el fichero y mantiene el canal abierto mientras el stream esté vivo, así que hay que declararlo en un `try-with-resources` para que se cierre. Con esta técnica se procesan ficheros de cualquier tamaño con memoria constante.

> ⚠️ **Error habitual**
>
> No cerrar el `Stream` de `Files.lines()`. Como implementa `AutoCloseable`, si no va en el `try(...)` el fichero queda bloqueado en Windows y no se puede borrar ni renombrar.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 1.6** del cuaderno de aula: filtrar los errores de un log con `Files.lines()` y contar las líneas por nivel.

Errores habituales del bloque de texto

> ⚠️ **Los cuatro fallos que más caen en examen**
>
> - **No indicar el charset** → tildes rotas al cambiar de sistema. Solución: siempre `StandardCharsets.UTF_8`.
> - **Olvidar** `newLine()` → todo pegado en una línea.
> - **Usar** `readAllLines()` **o** `readString()` **en ficheros enormes** → `OutOfMemoryError`. Solución: `Files.lines()`.
> - **No cerrar el** `Stream` **de** `Files.lines()` → fichero bloqueado.

Comparativa: cuándo usar cada herramienta de texto

HERRAMIENTA	ÚSALA PARA	CUIDADO
`BufferedReader` / `BufferedWriter`	Leer/escribir línea a línea, control fino.	Verboso; recuerda el charset y `newLine()`.
`Scanner`	Leer campos de un CSV ya convertidos a su tipo.	Configura `useLocale` para los decimales.
`PrintWriter`	Escribir informes con formato (`printf`).	No lanza `IOException`: usa `checkError()`.
`Files.readString` / `writeString`	Ficheros pequeños/medianos de una vez.	No apto para ficheros enormes.
`Files.lines()`	Ficheros enormes en streaming.	Ciérralo con try-with-resources.

🧠 Comprueba que sabes

1. ¿Por qué hay que indicar siempre el charset, y qué pasa si no lo haces?
2. ¿Qué devuelve `readLine()` al llegar al final del fichero?
3. ¿Cuándo usarías `Files.readString()` y cuándo `Files.lines()`?
4. ¿Por qué hay que cerrar el `Stream` de `Files.lines()` y no el de `readAllLines()`?

✅ Ya puedes hacer

- Ejercicios del bloque 1: ficheros de texto y NIO.2 (validador de CSV, informe con `PrintWriter`, analizador de logs con streams).

4 Ficheros binarios y serialización de objetos

El texto es legible, pero ineficiente para datos numéricos: guardar el número `2147483647` en texto ocupa 10 caracteres y obliga a parsearlo al leerlo. En binario, ese mismo entero son **4 bytes** exactos y se lee sin conversión. En este bloque guardamos datos primitivos con `DataOutputStream`, los leemos detectando el final con `EOFException`, saltamos a cualquier posición con `RandomAccessFile` (acceso aleatorio, **CE1.b**) y guardamos objetos enteros con la **serialización**. Criterios del bloque: **CE1.b, CE1.c, CE1.d** y **CE1.f**.

4.1 Escribir datos tipados: `DataOutputStream`

`DataOutputStream` decora un flujo de bytes y escribe tipos primitivos en su formato de máquina, siempre con el mismo tamaño. Así se consigue el binario compacto.

MÉTODO	BYTES	QUÉ ESCRIBE
`writeInt(int)`	4	Un entero de 32 bits.
`writeDouble(double)`	8	Un decimal de 64 bits (IEEE 754).
`writeBoolean(boolean)`	1	`0x01` si `true`, `0x00` si `false`.
`writeUTF(String)`	2 + N	2 bytes con la longitud + los N bytes del texto en UTF-8.

GuardarSensores.java — JAVA

```
import java.io.*;

public class GuardarSensores {
    public static void main(String[] args) {
        File f = new File("datos/telemetria.dat");
        try (DataOutputStream dos = new DataOutputStream(
                new BufferedOutputStream(new FileOutputStream(f)))) {
            // Cada sensor: id (int) + nombre (UTF) + temperatura (double) + alarma (boolean)
            dos.writeInt(101); dos.writeUTF("Caldera Central");
            dos.writeDouble(84.5); dos.writeBoolean(false);

            dos.writeInt(102); dos.writeUTF("Turbina 01");
            dos.writeDouble(102.3); dos.writeBoolean(true);
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
        // El tamaño se consulta DESPUÉS de cerrar: antes, parte sigue en el búfer
        System.out.println("Tamaño: " + f.length() + " bytes");
    }
}
```

Cada registro se escribe en el mismo orden de tipos. El tamaño se consulta al final: con el `BufferedOutputStream`, hasta que no se cierra el flujo, parte de los datos sigue en memoria y el fichero parece más pequeño.

> ⚠️ **Error habitual**
>
> Consultar `f.length()` dentro del `try`, antes de cerrar. Con búfer, el fichero aún no tiene todos los bytes y el tamaño sale mal (o 0).

> 💻 **Actividad de aula**
>
> Programa la **Actividad 2.1** del cuaderno de aula: grabar la telemetría de tres sensores en binario y comprobar el tamaño exacto del fichero.

4.2 Leer datos tipados y detectar el final: `DataInputStream` y `EOFException`

`DataInputStream` lee en el **mismo orden** en que se escribió: `readInt()`, `readUTF()`, `readDouble()`, `readBoolean()`. Pero surge un problema: ¿cómo sabemos que hemos llegado al final? En texto, `readLine()` devolvía `null`. Aquí no sirve: `readInt()` devuelve un entero, y `-1` podría ser un dato válido (un sensor a −1 ºC). Por eso, al intentar leer más allá del final, estos métodos **lanzan** `EOFException`. El patrón canónico es un bucle infinito que se corta al capturarla.

LeerSensores.java — JAVA

```
import java.io.*;

public class LeerSensores {
    public static void main(String[] args) {
        File f = new File("datos/telemetria.dat");
        int total = 0;
        try (DataInputStream dis = new DataInputStream(
                new BufferedInputStream(new FileInputStream(f)))) {
            while (true) {                      // bucle infinito controlado por EOF
                int id = dis.readInt();
                String nombre = dis.readUTF();
                double temp = dis.readDouble();
                boolean alarma = dis.readBoolean();
                System.out.printf("[%d] %s: %.1f%s%n",
                        id, nombre, temp, alarma ? " ¡ALARMA!" : "");
                total++;
            }
        } catch (EOFException e) {
            System.out.println("Fin del fichero: " + total + " sensores.");
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

La `EOFException` no es un error: es la **señal de fin**. Por eso se captura fuera del bucle y se usa para el mensaje final. Usar una excepción como final de fichero es el criterio **CE1.f** aplicado a binario.

> ⚠️ **Error habitual**
>
> Leer en distinto orden del que se escribió (p. ej. `readDouble()` antes que `readInt()`). El puntero lee bytes desfasados y salen números sin sentido o una `UTFDataFormatException`. **El orden de lectura debe ser idéntico al de escritura.**

> 💻 **Actividad de aula**
>
> Programa la **Actividad 2.2** del cuaderno de aula: leer el fichero de telemetría hasta el final detectando `EOFException`.

4.3 Acceso aleatorio: `RandomAccessFile`

Hasta ahora el acceso era secuencial: del principio al final. `RandomAccessFile` (RAF) cambia las reglas: trata el fichero como un **array de bytes** y permite mover un puntero a cualquier posición con `seek()`. Así puedes modificar el registro 500 sin tocar los otros 499. Es el acceso aleatorio del criterio **CE1.b**.

MÉTODO	QUÉ HACE
`new RandomAccessFile(f, "rw")`	Abre en lectura/escritura (`"r"` = solo lectura).
`seek(long pos)`	Mueve el puntero al byte indicado.
`getFilePointer()`	Devuelve la posición actual del puntero.
`length()` / `setLength(n)`	Tamaño del fichero / truncar o agrandar.

El truco está en los **registros de longitud fija**: si todos los registros miden lo mismo, la posición de cualquiera se calcula con una multiplicación. Para un registro de 36 bytes (id 4 + apellido 10 caracteres×2 + departamento 4 + salario 8):

Fórmula del offset — TEXTO

```
inicio del registro i       = i * 36
posición del salario en i   = i * 36 + 28     (tras id=4 y apellido=20)
```

AlmacenRAF.java JAVA

```java
import java.io.*;

public class AlmacenRAF {
    static final int TAM = 36;          // tamaño fijo de cada registro
    static final int OFF_SALARIO = 28;  // desplazamiento del salario dentro del registro

    public static void main(String[] args) throws IOException {
        File f = new File("datos/empleados.dat");
        // Modificar SOLO el salario del empleado en la posición 1, sin leer los demás
        try (RandomAccessFile raf = new RandomAccessFile(f, "rw")) {
            long offset = (1L * TAM) + OFF_SALARIO;
            raf.seek(offset);            // saltamos directo al salario del registro 1
            raf.writeDouble(2350.0);     // pisamos solo esos 8 bytes
            System.out.println("Salario cambiado en el byte " + offset);
        }
    }
}
```

Con `seek()` saltamos directo a los 8 bytes del salario y los sobrescribimos, sin tocar el resto del fichero. Esto sería imposible con los flujos secuenciales.

> ⚠️ **Error habitual**
>
> Guardar el apellido con `writeUTF()` (tamaño variable) en un fichero de registros fijos. En cuanto un texto mide distinto, los registros dejan de estar alineados y todas las fórmulas de `seek()` fallan. Para registros fijos se usa `writeChars()` rellenando a una longitud constante.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 2.3** del cuaderno de aula: gestionar un almacén de registros fijos y modificar un stock in situ con `seek()`.

4.4 Serialización de objetos

Escribir campo a campo está bien para datos simples, pero ¿y si queremos guardar un objeto entero con todos sus atributos? La **serialización** convierte un objeto vivo de la RAM en una secuencia de bytes, y la *deserialización* lo reconstruye. Java lo hace solo, con tres requisitos:

- La clase implementa `Serializable` (una interfaz marcadora, sin métodos).
- Declara `private static final long serialVersionUID = 1L;` como número de versión.
- Los campos que no deban guardarse (contraseñas, tokens) se marcan `transient`: al leer, vuelven a su valor por defecto (`null`, `0`).

Usuario.java JAVA

```java
import java.io.Serializable;

public class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String nombre;
    private transient String token;   // NO se guarda en disco

    public Usuario(int id, String nombre, String token) {
        this.id = id; this.nombre = nombre; this.token = token;
    }
    public String toString() {
        return id + " " + nombre + " token=" + token;
    }
}
```

Serializacion.java JAVA

```java
import java.io.*;

public class Serializacion {
    public static void main(String[] args) {
        File f = new File("datos/usuarios.ser");
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(f))) {
            oos.writeObject(new Usuario(1, "Elena", "SECRETO-123"));
        } catch (IOException e) { System.err.println(e.getMessage()); }

        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(f))) {
            Usuario u = (Usuario) ois.readObject();   // hace falta el cast
            System.out.println(u);                    // token sale null: era transient
        } catch (IOException | ClassNotFoundException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

Al recuperar el objeto, `token` vale `null`: por ser `transient`, nunca llegó al disco. `readObject()` devuelve un `Object` genérico, así que hay que hacer el *cast* a `Usuario`.

> 🏭 **En un entorno profesional**
>
> `transient` es tu aliado con el RGPD: marca así las contraseñas, tokens y datos sensibles para que jamás se escriban en un fichero de respaldo. Y declara siempre `serialVersionUID`: sin él, en cuanto cambies la clase, no podrás leer los ficheros antiguos (`InvalidClassException`).

> **💻 Actividad de aula**
>
> Programa la **Actividad 2.4** del cuaderno de aula: serializar dos usuarios y comprobar que el campo `transient` no se guarda.

4.5 Añadir objetos a un fichero existente

Hay una trampa clásica al querer **añadir** objetos a un fichero que ya tiene. Al crear un `ObjectOutputStream`, su constructor escribe una *cabecera mágica* de 4 bytes (`0xACED0005`). Si abres el fichero en modo append y creas otro `ObjectOutputStream`, escribe una **segunda cabecera en mitad del fichero**. Al leer, tras el primer objeto, el lector se encuentra esa cabecera donde esperaba un objeto y estalla:

El fallo — TEXTO

```
java.io.StreamCorruptedException: invalid type code: AC
```

La solución de la industria es una subclase que **no escribe la cabecera**, sobrescribiendo `writeStreamHeader()` con un cuerpo vacío. Se usa la normal la primera vez (para escribir la cabecera inicial) y la subclase cuando el fichero ya tiene datos.

MiObjectOutputStream.java — JAVA

```java
import java.io.*;

public class MiObjectOutputStream extends ObjectOutputStream {
    public MiObjectOutputStream(OutputStream out) throws IOException { super(out); }

    @Override
    protected void writeStreamHeader() {
        // Vacío: la cabecera 0xACED0005 ya está al principio del fichero
    }
}
```

AnadirUsuario.java — JAVA

```java
import java.io.*;

public class AnadirUsuario {
    public static void guardar(File f, Usuario u) throws IOException {
        boolean existe = f.exists() && f.length() > 0;
        try (FileOutputStream fos = new FileOutputStream(f, true);   // true = append
             ObjectOutputStream oos = existe
                     ? new MiObjectOutputStream(fos)   // ya hay cabecera: no repetir
                     : new ObjectOutputStream(fos)) {  // fichero nuevo: cabecera normal
            oos.writeObject(u);
        }
    }
}
```

El operador ternario decide: si el fichero ya existe y tiene datos, usa `MiObjectOutputStream` (sin cabecera); si es nuevo, el normal. Prever y resolver esta corrupción es **CE1.f** aplicado a serialización.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 2.5** del cuaderno de aula: añadir un usuario en cada ejecución sin corromper el fichero de objetos.

Errores habituales del bloque binario

> ⚠️ **Lo que más falla**
>
> - **Leer en distinto orden** del que se escribió → datos corruptos.
> - **Medir el fichero antes de cerrar** → tamaño equivocado por el búfer.
> - **Texto de tamaño variable en registros fijos de RAF** → se descuadran los `seek()`.
> - **Append de objetos con `ObjectOutputStream` normal** → `StreamCorruptedException`.
> - **Omitir `serialVersionUID`** → `InvalidClassException` al cambiar la clase.

Comparativa: las tres formas de binario

HERRAMIENTA	ACCESO	ÚSALA PARA
`DataOutputStream` / `DataInputStream`	Secuencial	Guardar muchos registros de tipos primitivos, compactos.
`RandomAccessFile`	Aleatorio	Modificar o consultar un registro concreto sin recorrer el resto.
`ObjectOutputStream` / `ObjectInputStream`	Secuencial	Guardar objetos completos con todos sus atributos.

> 🧠 **Comprueba que sabes**
>
> 1. ¿Por qué un binario ocupa menos que el mismo dato en texto?
> 2. ¿Cómo se detecta el fin de un fichero binario leído con `DataInputStream`?
> 3. ¿Qué permite `RandomAccessFile` que no permiten los flujos secuenciales?
> 4. ¿Para qué sirve `transient` y qué valor recupera un campo así al deserializar?
> 5. ¿Por qué falla el append normal de objetos y cómo se resuelve?

> ✅ **Ya puedes hacer**
>
> - Ejercicios del bloque 2: binarios, RAF y serialización (caja negra de flota, almacén con RAF, auditoría con append seguro).

5 Ficheros XML (DOM, SAX, StAX y JAXB)

El **XML** es el formato estándar para intercambiar datos entre empresas y sistemas: facturación electrónica (Facturae), mensajería bancaria (SEPA), servicios web, configuraciones. Java ofrece cuatro tecnologías para tratarlo, cada una para una necesidad distinta. Las veremos en orden, de la más intuitiva (DOM) a la más automática (JAXB). Criterios del bloque: **CE1.b, CE1.c, CE1.d** y **CE1.e**.

5.1 Fundamentos de XML y modelos de procesamiento

Un documento XML organiza la información en **etiquetas anidadas**. Cada etiqueta puede tener *atributos* y contener texto u otras etiquetas. Todo cuelga de una única *etiqueta raíz*:

catalogo.xml — XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
    <producto id="P101">
        <nombre>Disco SSD 1TB</nombre>
        <precio>79.99</precio>
    </producto>
</catalogo>
```

La primera línea es el *prólogo* (versión y codificación). `<catalogo>` es la raíz; `<producto>` tiene el atributo `id` y contiene las etiquetas hijas `<nombre>` y `<precio>`. Un XML *bien formado* cierra todas sus etiquetas y las anida correctamente.

Para procesarlo, Java ofrece cuatro enfoques. La primera decisión (qué modelo usar) es el criterio **CE1.b** aplicado a XML:

MODELO	CÓMO TRABAJA	CUÁNDO
DOM	Carga el árbol entero en RAM; se navega y modifica.	XML pequeños/medianos que hay que consultar o cambiar.
SAX	Lee por eventos, hacia adelante, sin cargar en memoria (*push*).	XML gigantes, solo lectura.
StAX	Lee por cursor que tú controlas y puedes parar (*pull*).	XML grandes donde buscas algo concreto.
JAXB	Mapea etiquetas a objetos con anotaciones, sin tocar nodos.	Convertir objetos ↔ XML de forma automática.

5.2 Leer y modificar XML con DOM

DOM (Document Object Model) carga todo el XML en memoria como un *árbol de nodos*: el `Document`, los `Element` (etiquetas), el texto y los atributos. Así puedes navegar en cualquier dirección y modificar valores.

INSTRUCCIÓN	QUÉ HACE
`DocumentBuilderFactory.newInstance().newDocumentBuilder()`	Crea el parser DOM.
`builder.parse(File)`	Lee el XML y construye el árbol en RAM.
`doc.getElementsByTagName("producto")`	Devuelve la `NodeList` de esas etiquetas.
`nodo.getNodeType() == Node.ELEMENT_NODE`	Filtra los nodos de texto (`#text`).
`elem.getAttribute("id")` / `getTextContent()`	Lee un atributo / el texto de la etiqueta.
`elem.setTextContent(s)`	Cambia el texto de la etiqueta.

SubirPrecios.java JAVA

```java
import java.io.File;
import java.util.Locale;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.*;

public class SubirPrecios {
    public static void main(String[] args) throws Exception {
        Document doc = DocumentBuilderFactory.newInstance()
                .newDocumentBuilder().parse(new File("datos/catalogo.xml"));
        doc.getDocumentElement().normalize();

        NodeList productos = doc.getElementsByTagName("producto");
        for (int i = 0; i < productos.getLength(); i++) {
            Node nodo = productos.item(i);
            if (nodo.getNodeType() == Node.ELEMENT_NODE) {   // ignora los #text
                Element prod = (Element) nodo;
                Element precio = (Element) prod.getElementsByTagName("precio").item(0);
                double nuevo = Double.parseDouble(precio.getTextContent()) * 1.10;
                precio.setTextContent(String.format(Locale.US, "%.2f", nuevo));
                System.out.println(prod.getAttribute("id") + " -> " + precio.getTextContent());
            }
        }
    }
}
```

Recorremos los `<producto>` y les subimos el precio un 10 % en memoria. El filtro `ELEMENT_NODE` es obligatorio.

> ⚠️ **Error habitual: los nodos `#text`**
>
> Los saltos de línea y sangrías entre etiquetas se leen como nodos de texto. Si haces el *cast* a `(Element)` sin comprobar antes `getNodeType() == Node.ELEMENT_NODE`, salta `ClassCastException`. Es la trampa número uno de DOM.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 3.1** del cuaderno de aula: leer un catálogo con DOM, esquivar los `#text` y subir los precios.

5.3 Crear XML con DOM y guardarlo con `Transformer`

DOM también sirve para *crear* un XML desde cero: se construye un documento vacío, se crean elementos con `doc.createElement()` y se enganchan con `appendChild()`. Para volcarlo a disco con sangría legible se usa un `Transformer`. Generar un XML a partir de datos es almacenar información (**CE1.d**) y, si el origen era otro formato, una conversión (**CE1.e**).

CrearXML.java JAVA

```java
import java.io.File;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.*;

public class CrearXML {
    public static void main(String[] args) throws Exception {
        Document doc = DocumentBuilderFactory.newInstance()
                .newDocumentBuilder().newDocument();

        Element raiz = doc.createElement("alumnos");     // etiqueta raíz
        doc.appendChild(raiz);

        Element alumno = doc.createElement("alumno");
        alumno.setAttribute("expediente", "EXP-01");     // atributo
        Element nombre = doc.createElement("nombre");
        nombre.setTextContent("Lucía Álvarez");
        alumno.appendChild(nombre);
        raiz.appendChild(alumno);

        // Volcado a disco con sangría de 4 espacios
        Transformer tf = TransformerFactory.newInstance().newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");
        tf.transform(new DOMSource(doc), new StreamResult(new File("datos/alumnos.xml")));
    }
}
```

> ⚠️ **Error habitual**
>
> Los elementos no se crean con `new Element()`: los fabrica el documento con `doc.createElement()`. Y si olvidas las propiedades de sangría del `Transformer`, el XML sale correcto pero en una sola línea ilegible.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 3.2** del cuaderno de aula: construir un XML de alumnos con DOM y volcarlo con sangría usando `Transformer`.

5.4 Leer XML por eventos con SAX

DOM carga todo en memoria: inviable para un XML de varios GB. **SAX** lee el documento como una cinta transportadora, de principio a fin, disparando *eventos* que atiende un `DefaultHandler`. El

consumo de memoria es mínimo y constante, sea el fichero de 1 KB o de 5 GB. La contrapartida: solo lee hacia adelante, no puede modificar ni volver atrás.

CALLBACK	CUÁNDO SE DISPARA
`startElement(...)`	Al abrir una etiqueta. Aquí se limpia el acumulador de texto.
`characters(...)`	Al leer texto. Se acumula en un `StringBuilder`.
`endElement(...)`	Al cerrar una etiqueta. Aquí se procesa el texto acumulado.

SumaPreciosSAX.java **JAVA**

```java
import java.io.File;
import javax.xml.parsers.SAXParserFactory;
import org.xml.sax.Attributes;
import org.xml.sax.helpers.DefaultHandler;

public class SumaPreciosSAX {
    static class Handler extends DefaultHandler {
        StringBuilder buffer = new StringBuilder();
        double suma = 0;

        public void startElement(String u, String l, String q, Attributes a) {
            buffer.setLength(0);                // nuevo texto: vaciamos el acumulador
        }
        public void characters(char[] ch, int start, int len) {
            buffer.append(ch, start, len);      // puede llegar en varios trozos
        }
        public void endElement(String u, String l, String q) {
            if (q.equalsIgnoreCase("precio")) {
                suma += Double.parseDouble(buffer.toString().trim());
            }
        }
        public void endDocument() {
            System.out.printf("Suma de precios: %.2f%n", suma);
        }
    }

    public static void main(String[] args) throws Exception {
        SAXParserFactory.newInstance().newSAXParser()
                .parse(new File("datos/catalogo.xml"), new Handler());
    }
}
```

> **⚠️ Error habitual**
>
> Hacer `parseDouble()` directamente dentro de `characters()`. El parser puede trocear el texto de una etiqueta en varias llamadas; hay que acumular en un `StringBuilder` y procesar en `endElement()`.

> **💻 Actividad de aula**
>
> Programa la **Actividad 3.3** del cuaderno de aula: calcular la suma y la media de precios del catálogo con SAX.

5.5 Leer XML en streaming con StAX

En SAX es el parser quien te empuja los eventos: no puedes pararlo. **StAX** te da el control: tú pides el siguiente evento con `next()` en un bucle, y puedes **parar cuando quieras**. Es ideal para buscar un dato concreto en un XML grande sin leerlo entero. Comparar SAX (push) frente a StAX (pull) y elegir es criterio **CE1.b**.

MÉTODO	QUÉ HACE
`hasNext()` / `next()`	¿Quedan eventos? / avanza y devuelve el tipo del siguiente.
`getLocalName()`	Nombre de la etiqueta actual.
`getAttributeValue(null, "id")`	Valor de un atributo.
`getText()` / `isWhiteSpace()`	Texto del evento / si es solo espacio en blanco.

BuscarStAX.java JAVA

```java
import java.io.FileInputStream;
import java.io.InputStream;
import javax.xml.stream.*;

public class BuscarStAX {
    public static void main(String[] args) throws Exception {
        String idBuscado = "P101";
        try (InputStream in = new FileInputStream("datos/catalogo.xml")) {
            XMLStreamReader r = XMLInputFactory.newInstance().createXMLStreamReader(in);
            String etiqueta = "";
            boolean dentro = false, encontrado = false;

            while (r.hasNext() && !encontrado) {         // paramos en cuanto lo hallamos
                int evento = r.next();
                if (evento == XMLStreamConstants.START_ELEMENT) {
                    etiqueta = r.getLocalName();
                    if (etiqueta.equals("producto"))
                        dentro = idBuscado.equals(r.getAttributeValue(null, "id"));
                } else if (evento == XMLStreamConstants.CHARACTERS
                           && dentro && !r.isWhiteSpace()) {
                    if (etiqueta.equals("nombre"))
                        System.out.println("Encontrado: " + r.getText());
                } else if (evento == XMLStreamConstants.END_ELEMENT
                           && r.getLocalName().equals("producto") && dentro) {
                    encontrado = true;                   // salida anticipada
                }
            }
            r.close();   // XMLStreamReader NO es AutoCloseable
        }
    }
}
```

El bucle se corta con `!encontrado` en cuanto localiza el producto: no lee el resto del documento. Esa salida anticipada es lo que distingue a StAX.

> ⚠️ **Error habitual**
>
> Guardar el texto de todos los eventos `CHARACTERS` sin comprobar `isWhiteSpace()`, y olvidar `r.close()`: `XMLStreamReader` no es `AutoCloseable`, hay que cerrarlo a mano.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 3.4** del cuaderno de aula: buscar un producto con StAX y dejar de leer en cuanto aparece.

5.6 Mapeo objeto-XML con JAXB

Construir el XML nodo a nodo con DOM es laborioso. **JAXB** (Jakarta XML Binding) lo automatiza: anotas una clase POJO y JAXB convierte objeto↔XML por ti. *Marshalling* = objeto a XML; *unmarshalling* = XML a objeto. Es la forma más directa de convertir entre objetos y XML (**CE1.d** y **CE1.e**).

ANOTACIÓN	QUÉ HACE
`@XmlRootElement(name="curso")`	Marca la clase como etiqueta raíz.
`@XmlAttribute`	El campo se guarda como atributo.
`@XmlElement`	El campo se guarda como etiqueta hija.

Curso.java — JAVA

```java
import jakarta.xml.bind.annotation.*;

@XmlRootElement(name = "curso")
@XmlAccessorType(XmlAccessType.FIELD)
public class Curso {
    @XmlAttribute private String codigo;
    @XmlElement   private String nombre;
    @XmlElement   private int horas;

    public Curso() {}   // constructor vacío OBLIGATORIO para JAXB
    public Curso(String nombre, int horas, String codigo) {
        this.nombre = nombre; this.horas = horas; this.codigo = codigo;
    }
}
```

MapeoJAXB.java — JAVA

```java
import java.io.File;
import jakarta.xml.bind.*;

public class MapeoJAXB {
    public static void main(String[] args) throws Exception {
        File f = new File("datos/curso.xml");
        JAXBContext ctx = JAXBContext.newInstance(Curso.class);

        Marshaller m = ctx.createMarshaller();                  // objeto -> XML
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.marshal(new Curso("Acceso a Datos", 140, "0486"), f);

        Unmarshaller um = ctx.createUnmarshaller();             // XML -> objeto
        Curso c = (Curso) um.unmarshal(f);
        System.out.println("Recuperado: " + c.getClass().getSimpleName());
    }
}
```

Sin crear un solo nodo, JAXB genera el XML y lo vuelve a leer como objeto. Necesita un **constructor vacío** para poder instanciar la clase por reflexión, y en el proyecto, las dependencias de JAXB en el `pom.xml`.

> ⚠️ **Error habitual**
>
> Olvidar el constructor vacío `public Curso() {}`: JAXB no puede instanciar el objeto al leer y falla en ejecución.

> 💻 **Actividad de aula**
>
> Programa la **Actividad 3.5** del cuaderno de aula: convertir un objeto `Curso` a XML y recuperarlo con JAXB.

Errores habituales del bloque XML

> ⚠️ **Lo que más falla**
>
> - **Cast a** `(Element)` **sin filtrar** `#text` en DOM → `ClassCastException`.
> - **Olvidar la sangría del** `Transformer` → XML en una sola línea.
> - `parseDouble()` **dentro de** `characters()` en SAX → número troceado.
> - **No cerrar** `XMLStreamReader` en StAX (no es `AutoCloseable`).
> - **Falta el constructor vacío** en la clase JAXB.

Comparativa: las cuatro tecnologías XML

TECNOLOGÍA	MEMORIA	DIRECCIÓN	MODIFICA	IDEAL PARA
DOM	Todo en RAM	Libre	Sí	XML pequeños que hay que editar.
SAX	Mínima	Solo adelante (push)	No	XML gigantes, leer todo.
StAX	Mínima	Adelante, controlado (pull)	No	Buscar un dato en un XML grande.
JAXB	Objeto en RAM	—	Sí (vía objeto)	Convertir objetos ↔ XML.

🧠 Comprueba que sabes

1. ¿Por qué DOM no sirve para un XML de 5 GB y SAX sí?
2. ¿Qué diferencia a SAX (push) de StAX (pull)?
3. ¿Por qué hay que filtrar los nodos `#text` al recorrer un árbol DOM?
4. ¿Qué dos cosas necesita una clase para que JAXB la convierta a XML?

✅ Ya puedes hacer

- Ejercicios del bloque 3: ecosistema XML (factura con DOM, modificador de tarifas, analizador SAX/StAX, mapeo JAXB).

6 Integración: una aplicación multi-formato

Cada formato lo hemos visto por separado. En una aplicación real se combinan: se leen datos de un CSV, se guardan en binario para el día a día y se exportan a XML para intercambiarlos. Aquí juntamos los tres bloques y, además, aprendemos a **probar y documentar** lo que hacemos.

6.1 Convertir un catálogo entre CSV, binario y XML

Imagina un catálogo de productos que llega en CSV de un TPV antiguo. Queremos: (1) leerlo y validarlo, (2) guardarlo en binario compacto para consultas rápidas y (3) exportarlo a XML para un proveedor. Cada paso usa lo aprendido en su bloque. Una buena arquitectura separa el *modelo* (la clase `Producto`) de los *servicios* de persistencia (uno por formato).

Producto.java JAVA

```java
import java.io.Serializable;

/** Modelo del catálogo. Serializable para poder guardarlo con ObjectOutputStream. */
public class Producto implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String ref;
    private final String nombre;
    private final double precio;
    private transient double costeInterno;   // dato sensible: no se exporta

    public Producto(String ref, String nombre, double precio) {
        this.ref = ref; this.nombre = nombre; this.precio = precio;
    }
    public String getRef()    { return ref; }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
}
```

CatalogoService.java JAVA

```java
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.util.*;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.*;

public class CatalogoService {

    /** 1. TEXTO: lee el CSV validando cada línea (descarta las corruptas). */
    public List<Producto> importarCsv(Path csv) throws IOException {
        List<Producto> lista = new ArrayList<>();
        try (BufferedReader br = Files.newBufferedReader(csv, StandardCharsets.UTF_8)) {
            br.readLine();                                  // cabecera
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] c = linea.split(";");
                if (c.length == 3) {
                    try {
                        lista.add(new Producto(c[0], c[1], Double.parseDouble(c[2])));
                    } catch (NumberFormatException e) {
                        System.err.println("Línea descartada: " + linea);
                    }
                }
            }
        }
        return lista;
    }

    /** 2. BINARIO: guarda la lista completa serializada. */
    public void exportarBinario(List<Producto> productos, Path dat) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(
                Files.newOutputStream(dat))) {
            oos.writeObject(productos);
        }
    }

    /** 3. XML: genera un catálogo con DOM y Transformer. */
    public void exportarXml(List<Producto> productos, Path xml) throws Exception {
        Document doc = DocumentBuilderFactory.newInstance()
                .newDocumentBuilder().newDocument();
        Element raiz = doc.createElement("catalogo");
        doc.appendChild(raiz);
        for (Producto p : productos) {
            Element e = doc.createElement("producto");
            e.setAttribute("ref", p.getRef());
            Element nom = doc.createElement("nombre");
            nom.setTextContent(p.getNombre());
            Element pre = doc.createElement("precio");
            pre.setTextContent(String.valueOf(p.getPrecio()));
```

```
            e.appendChild(nom); e.appendChild(pre);
            raiz.appendChild(e);
        }
        Transformer tf = TransformerFactory.newInstance().newTransformer();
        tf.setOutputProperty(OutputKeys.INDENT, "yes");
        tf.transform(new DOMSource(doc), new StreamResult(xml.toFile()));
    }
}
```

Cada método es un servicio independiente que reutiliza una técnica de su bloque: `BufferedReader` para el CSV, serialización para el binario, DOM + `Transformer` para el XML. El `main` los encadena: importar → guardar en binario → exportar a XML, capturando las excepciones de cada paso. Esto integra los criterios **CE1.a** a **CE1.f**.

6.2 Probar y documentar la aplicación

Una aplicación no está terminada cuando «funciona»: lo está cuando **se puede comprobar que funciona** y **otra persona la entiende**. Esto es el criterio **CE1.g**, y tiene dos patas.

Pruebas automáticas con JUnit 5. En vez de mirar la salida por consola a ojo, escribimos pruebas que comprueban el resultado y salen en verde o en rojo. Cada prueba crea sus propios datos (con `@TempDir`, una carpeta temporal) para no depender de ficheros reales que alguien podría cambiar.

CatalogoServiceTest.java — JAVA

```
import static org.junit.jupiter.api.Assertions.*;
import java.nio.file.*;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

class CatalogoServiceTest {
    @TempDir Path carpeta;              // JUnit la crea y la borra en cada prueba

    @Test
    void importaSoloLineasValidas() throws Exception {
        Path csv = carpeta.resolve("p.csv");
        Files.writeString(csv, "REF;NOMBRE;PRECIO\nA-1;Uno;10.0\nB-2;Malo;xxx\n");
        List<Producto> r = new CatalogoService().importarCsv(csv);
        assertEquals(1, r.size());     // la línea con precio "xxx" se descarta
        assertEquals("A-1", r.get(0).getRef());
    }
}
```

Documentación con Javadoc y README. Los comentarios `/** ... */` sobre clases y métodos (con `@param`, `@return`, `@throws`) generan documentación navegable y aparecen al pasar el ratón en el IDE. El `README.md` del proyecto explica qué hace, cómo se ejecuta y cómo se prueban.

🏭 En un entorno profesional

Ninguna empresa acepta código sin pruebas ni documentación. Las pruebas evitan que un cambio rompa lo que ya funcionaba (regresiones); el Javadoc y el README permiten que otra persona (o tú dentro de seis meses) retome el proyecto sin descifrarlo línea a línea.

💻 Actividad de aula

Programa la **Actividad 4.1** del cuaderno de aula: escribir pruebas JUnit de un método de lectura, documentar con Javadoc y redactar el README del proyecto.

✅ Ya puedes hacer

- Práctica final de la unidad: gestor multi-formato de catálogo, que integra los tres formatos, el control de excepciones, las pruebas y la documentación.

7 Visión de conjunto

Buenas prácticas que valen para toda la unidad

Distinguiendo tres niveles:

- **Obligatorio para que funcione:** cerrar siempre los recursos (try-with-resources); en binario, leer en el mismo orden que se escribió; en DOM, filtrar los nodos `#text`.
- **Muy recomendable:** indicar siempre el charset (`UTF_8`) en texto; declarar `serialVersionUID`; envolver los flujos en su versión *Buffered*; construir rutas con `Path`, nunca con barras a mano.
- **Buena práctica profesional:** separar el modelo de los servicios de persistencia; marcar los datos sensibles como `transient`; capturar las excepciones de forma diferenciada; acompañar el código de pruebas y documentación.

¿Qué formato elijo? Comparativa entre bloques

NECESITO…	FORMATO Y HERRAMIENTA
Que lo lea una persona o un Excel	Texto / CSV con `BufferedReader` o `Scanner`.
Máxima compactación y velocidad para el programa	Binario con `DataOutputStream`.
Modificar un registro concreto sin recorrer el resto	Binario con `RandomAccessFile`.
Guardar objetos completos tal cual	Serialización con `ObjectOutputStream`.
Intercambiar datos con otra empresa o sistema	XML (DOM/JAXB para crear; SAX/StAX para leer grandes).

> 💡 **La pregunta que resume la unidad**
>
> Ante un problema nuevo: *¿quién lee los datos (persona o programa)?*, *¿necesito acceso directo a un registro?* y *¿voy a intercambiarlos con otro sistema?*. Las respuestas te llevan directas al formato y a la clase de Java.

8 Resumen y conceptos que debes dominar

Las ideas clave, en una ojeada

- Un fichero es una secuencia de bytes; lo que cambia es cómo los interpretamos (texto/binario) y en qué orden los recorremos (secuencial/aleatorio).
- Todo pasa por flujos con búfer y se cierra con try-with-resources.
- Texto: charset siempre UTF-8; `BufferedReader` / `Writer`, `Scanner`, `PrintWriter` y NIO.2 (`Files`).
- Binario: compacto; `Data*Stream` secuencial, `RandomAccessFile` aleatorio, serialización para objetos.
- XML: DOM (editar), SAX (leer enorme), StAX (buscar y parar), JAXB (objeto↔XML).
- Una aplicación se termina con excepciones controladas, pruebas y documentación.

Conceptos que debes dominar, por criterio de evaluación

Después de esta unidad deberías ser capaz de…

CE1.a · Gestión de ficheros y directorios

- ✔️ Crear, listar, mover y borrar ficheros y carpetas con `Path` y `Files`.
- ✔️ Construir rutas portables con `Path.of()`.

CE1.b · Formas de acceso

- ✔️ Distinguir texto de binario y acceso secuencial de aleatorio, y elegir el adecuado.
- ✔️ Comparar DOM/SAX/StAX y saber cuándo usar cada uno.

CE1.c · Recuperar información

- ✔️ Leer texto con `BufferedReader`, `Scanner` y `Files.lines()`.
- ✔️ Leer binario con `DataInputStream` y objetos con `ObjectInputStream`.
- ✔️ Leer XML con DOM, SAX y StAX.

CE1.d · Almacenar información

- ✔️ Escribir texto con `BufferedWriter`, `PrintWriter` y `Files.writeString()`.
- ✔️ Escribir binario con `DataOutputStream` y objetos con `ObjectOutputStream`.
- ✔️ Generar XML con DOM/`Transformer` y JAXB.

CE1.e · Conversiones entre formatos

- ✔️ Convertir entre codificaciones de caracteres.
- ✔️ Convertir datos entre texto, binario y XML.

CE1.f · Gestión de excepciones

- ✔️ Usar `try-with-resources` y capturar `IOException`.
- ✔️ Detectar el fin de fichero con `EOFException` y resolver la corrupción del append.

CE1.g · Probar y documentar

- ✔️ Escribir pruebas JUnit independientes de los datos reales.
- ✔️ Documentar con Javadoc y README.

9 Glosario y preguntas de repaso

Glosario

- **Persistencia**: guardar datos en un soporte para que sobrevivan al cierre del programa.
- **Flujo (stream)**: canal por el que circulan los datos entre el programa y el fichero.
- **Búfer**: zona de RAM que acumula datos para leer/escribir en bloques y no byte a byte.
- **Charset**: tabla que traduce entre caracteres y bytes (ASCII, ISO-8859-1, UTF-8).
- **Mojibake**: caracteres corruptos por leer con un charset distinto del de escritura.
- **Acceso secuencial / aleatorio**: recorrer el fichero en orden / saltar a cualquier posición.
- **Serialización**: convertir un objeto de la RAM en bytes; deserialización, lo inverso.
- **transient**: modificador que excluye un campo de la serialización.
- **EOFException**: excepción que señala el fin de un fichero binario.
- **DOM / SAX / StAX / JAXB**: las cuatro tecnologías de Java para procesar XML.
- **Marshalling**: en JAXB, convertir un objeto en XML (unmarshalling, al revés).
- **try-with-resources**: sintaxis que cierra los recursos automáticamente.

Preguntas de repaso

Comprueba si dominas la unidad. No hay respuestas aquí: repásalas con los apuntes.

1. Explica con tus palabras la diferencia entre un fichero de texto y uno binario, con un ejemplo de cuándo usarías cada uno.
2. ¿Por qué es imprescindible indicar el charset al leer y escribir texto? ¿Qué error se produce si no lo haces?
3. ¿Qué ventaja aporta el búfer y cómo se añade mediante el patrón decorador?
4. Compara `Files.readString()` y `Files.lines()`: ¿cuándo es peligroso el primero?
5. ¿Cómo se detecta el final de un fichero binario leído con `DataInputStream`, y por qué no vale con devolver −1?
6. ¿Qué permite `RandomAccessFile` que no permite `DataInputStream`? Pon un caso de uso.
7. Un compañero añade objetos a un `.ser` y al leer obtiene `StreamCorruptedException: invalid type code: AC`. ¿Qué ha pasado y cómo se arregla?
8. ¿Para qué sirve `transient`? Da un ejemplo real relacionado con la seguridad.
9. ¿Por qué DOM no es viable para un XML de varios GB? ¿Qué usarías en su lugar y por qué?
10. ¿Qué diferencia práctica hay entre SAX y StAX a la hora de buscar un dato concreto?
11. ¿Qué dos requisitos mínimos tiene una clase para que JAXB la convierta a XML?
12. Tienes que leer un CSV, guardarlo en binario y exportarlo a XML. Indica qué clase de Java usarías en cada paso.
13. ¿Por qué una prueba JUnit debe crear sus propios datos en vez de leer un fichero de `datos/`?

14. Enumera tres buenas prácticas «obligatorias para que funcione» vistas en la unidad.

Anexo. Tabla de trazabilidad

Dónde se trabaja cada criterio de evaluación: apartado de los apuntes, actividad de aula y ejercicio o práctica.

CE	APARTADOS	ACTIVIDADES DE AULA	EJERCICIOS / PRÁCTICA
CE1.a	2.2, 3.5, 6.1	0.1, 1.5	Bloque 1 · Práctica final
CE1.b	2.1, 4.3, 5.5, 6.1	2.3, 3.4	Bloque 2 · Bloque 3 · Práctica final
CE1.c	2.3, 2.4, 3.1–3.6, 4.2–4.4, 5.1–5.5, 6.1	1.1–1.6, 2.2–2.4, 3.1, 3.3, 3.4	Bloques 1, 2 y 3 · Práctica final
CE1.d	2.3, 2.4, 3.1–3.5, 4.1, 4.3–4.5, 5.3, 5.6, 6.1	1.1–1.5, 2.1, 2.4, 2.5, 3.2, 3.5	Bloques 1, 2 y 3 · Práctica final
CE1.e	3.2, 5.3, 5.6, 6.1	1.2, 3.2, 3.5	Bloque 3 · Práctica final
CE1.f	2.5, 4.2, 4.5, 6.1	2.2, 2.5	Bloques 1, 2 y 3 · Práctica final
CE1.g	6.2	4.1	Práctica final